

Workshop 3 Fonctions

Vous venez d'être recruté(e) en tant que développeur(se) junior dans une startup spécialisée en solutions de santé digitale. Votre première mission est critique : développer un prototype d'un système de gestion d'hôpital qui sera présenté à des investisseurs.

Votre chef de projet vous demande un système simple, mais fonctionnel, permettant de gérer les patients, les médecins et certaines opérations internes.

Votre Mission

Développer un mini système en Python permettant de gérer :

- Les patients
- Les médecins
- Les services de l'hôpital
- Quelques traitements de données

Étape 1 : Gestion des patients

1. Créer une **liste patients** contenant au moins 5 patients.
Chaque patient est un dictionnaire avec :
 - `id` (int)
 - `nom` (str)
 - `age` (int)
 - `maladie` (str) {Grippe, Diabete, Asthme, Cardiaque, Infection}
 - `etat` (str : "critique", "stable")
2. Afficher tous les patients (boucle).
3. Ajouter un nouveau patient à la liste.
4. Supprimer un patient à partir de son `id`.
5. Modifier l'état d'un patient (ex : de "stable" à "critique").

Étape 2 : Gestion des médecins

1. Créer une **liste medecins** avec au moins 3 médecins.
Chaque médecin contient :
 - `id`
 - `nom`
 - `specialite` {Cardiologie, Neurologie, Generaliste}
 - `patients` → liste des IDs des patients
2. Assigner un patient à un médecin (ajouter l'ID du patient).
3. Afficher chaque médecin avec ses patients.
4. Trouver quel médecin suit un patient donné.

Étape 3 : Services de l'hôpital

1. Créer un tuple `services` contenant :
("Urgence", "Cardiologie", "Neurologie", "Pédiatrie".)
2. Afficher tous les services.
3. Vérifier si un service existe (ex : "Cardiologie").

Étape 4 : Modularisation avec fonctions

Créer les fonctions suivantes :

- `ajouter_patient(patients, patient)`
- `afficher_patients(patients)`
- `filtrer_par_etat(patients, etat)`
- `assigner_patient(medecins, patient_id, medecin_id)`
- `compter_patients_par_medecin(medecins)`

-> Chaque fonction doit être testée.

Étape 5 : Traitement avancé

1. Extraire les patients majeurs (`age > 18`).

2. Créer une liste contenant uniquement les **noms des patients**.
3. Filtrer les patients en état "**critique**".
4. Augmenter l'âge de tous les patients de +1 (simulation).
5. Trier les patients selon leur âge.

-> Essayez d'utiliser :

- `filter()`
- `map()`
- `lambda`

Étape 6 : Système final (Simulation)

Objectif : Intégrer toutes les parties

Créer un menu simple :

1. Ajouter un patient
2. Afficher les patients
3. Assigner un patient à un médecin
4. Voir les patients critiques
5. Quitter

Le programme doit :

- Lire le choix utilisateur
- Exécuter l'action correspondante
- Boucler jusqu'à "Quitter"

Livrable attendu

Vous devez présenter :

- Un code structuré (fonctions)
- Utilisation des listes, dictionnaires, tuples
- Manipulation des données (filter/map implicite)
- Un système fonctionnel en console